

CSC111 Project Proposal: Charting Your Academic Path – A Graph-Based Course Planning Tool for UofT Students

Efren Medina Arias, Jack Anderson, Diego Figarella, Tanish Ariyur

March 31, 2026

Problem Description and Research Question

The University of Toronto offers hundreds of courses across its Faculty of Arts and Science, many of which have long chains of prerequisite and corequisite requirements. As students at UofT, planning our paths toward our degrees is often tedious and confusing. A student who wants to eventually enrol in an upper-year course must work backwards through multiple layers of prerequisites, often discovering gaps in their plan only after the fact. This adds unnecessary stress and uncertainty to academic planning, regardless of what program or discipline a student is pursuing.

This problem can be modelled in two ways. The first is a directed graph, where each course is a vertex and each prerequisite relationship is a directed edge from the required course to the one it unlocks. This representation is intuitive and closely mirrors how the calendar describes prerequisites, making it natural for storing and querying the full course catalogue. The second is a search tree, where each node represents a student state, storing the set of courses completed so far, and each branch represents the decision to take a new eligible course. This tree-based approach lets us systematically explore all possible sequences a student could follow and find every valid path to a target course, similar to how a game tree explores all possible moves in a strategy game. We are still trying to figure out the best choice due to the massive data set we have. We are currently doing research and would highly appreciate some feedback.

Given a student's completed courses and a set of target courses they wish to take, our program will determine all possible sequences of courses the student could take to satisfy all prerequisite and corequisite requirements, and display the resulting paths in an interactive visual format so the student can choose the option that best fits their plan.

Computational Plan

Data Source

Our primary data source is the University of Toronto Faculty of Arts and Science 2025–2026 Academic Calendar. We will work with an HTML version of the calendar, which contains for each course its course code, title, description, prerequisite requirements, corequisite requirements, and exclusion notices. A representative example from the calendar is:

CSC236H1 – Introduction to the Theory of Computation.

Prerequisite: (60% or higher in CSC111H1) and (60% or higher in MAT102H5 or MAT137Y1 or MAT157Y1)

We will use the Python library `beautifulsoup4` to parse the HTML calendar file and extract each course entry into a structured JSON file. Each entry in the JSON will store the course code, the raw prerequisite string, and the raw exclusion list. This gives us a clean, reusable dataset that we can load into our program without re-parsing the HTML each time.

Graph and Tree Model

We can use two different data structures to model the course planning problem.

For the graph model, we represent the full course catalogue as a directed graph where each vertex is a UofT course and each directed edge from course A to course B means that A is a prerequisite of B. Corequisites are stored as a distinct edge type, and course exclusions are stored as attributes on each vertex. Courses with OR prerequisites

are handled by storing multiple valid prerequisite combinations per vertex, so that satisfying any one combination is enough to unlock the course. This structure closely mirrors how the calendar describes course requirements and makes it straightforward to query what a student needs for any given course. Given a student's completed courses and a target, we can perform a traversal over this graph to find the set of courses that lie on the path between them.

For the tree model, we implement a custom `CourseTree` class to handle planning and path exploration. Each node in this tree represents a student state, storing the set of courses the student has completed and their total credit count. The current state determines which courses are available to take, and selecting a course expands the tree by adding a new child node reflecting the updated course history. This is analogous to how a game tree works: just as a game tree explores all possible sequences of moves from a given position, our course tree explores all possible sequences of courses a student could take from their current standing. Rather than finding a single path, this approach lets us enumerate all valid paths to a target course and present them to the student, who can then choose the sequence that best fits their schedule and interests.

Computations

We plan to implement the following core computations, organized by approach.

Using only the graph:

1. Parse the HTML calendar into a structured JSON dataset using `beautifulsoup4`, storing each course's code, prerequisite string, and exclusion list.
2. Build the directed prerequisite graph from the JSON dataset, handling AND and OR prerequisite logic as well as corequisites and exclusions.
3. Implement an eligibility check that, given a student's completed courses, traverses the graph to return all courses the student is currently eligible to enrol in, respecting prerequisite combinations and exclusion constraints.
4. Implement a path-finding method that performs a traversal over the graph to find all valid sequences of courses between the student's current state and a target course, presenting every possible route rather than just one.
5. Implement a credit-counting method that returns the number of credits a student has accumulated, with an optional filter by department, useful for checking whether specific program requirements are met.

Using only the tree:

1. Build the `CourseTree` by starting from the student's current state as the root and expanding child nodes by selecting eligible courses, with each node storing the updated set of completed courses and credit count.
2. Implement an eligibility check at each node that determines which courses are available to take next based on the prerequisites and exclusions encoded in the tree structure.
3. Implement a path-finding method that searches the tree for all root-to-node paths that result in a target course becoming eligible, returning every valid sequence so the student can choose the one that best fits their plan.
4. Implement a credit-counting method at each node that tracks accumulated credits, with an optional filter by department.

Visualization and Interaction

Our program will provide an interactive interface with two components.

For user input, students can enter their completed courses and one or more target courses via a command-line.

We will use an interactive graph display using **plotly** to render the relevant subgraph of the prerequisite network in the browser.

Nodes will be colour-coded: green for completed courses, yellow for courses on the recommended path, and red for target courses. Edges will be directed arrows indicating prerequisite flow.

New Python Libraries

We will use two external libraries in our project.

The first is `plotly`, which we will use as our primary visualization library. Plotly is appropriate for this project because it produces interactive, browser-based visuals that allow students to explore the course prerequisite network. Since we are building a custom graph data structure, we will extract node and edge positions directly from it and pass them into Plotly. We will create two `go.Scatter()` traces: one with `mode='lines'` to draw edges between courses, and one with `mode='markers'` and `hoverinfo='text'` to draw course nodes. Node colours are set through the `marker` parameter, allowing us to colour-code courses as green (completed), yellow (on the recommended path), or red (target). Both traces are combined using `go.Figure()` with a `go.Layout()` that hides axis gridlines and sets `hovermode='closest'`, so that hovering over a node displays that course's details. The result is rendered in the browser using `fig.show()`.

The second is `beautifulsoup4`, which we have already used to parse the UofT course calendar HTML file and extract structured course data. We used `beautifulsoup4`'s `find` method to locate course entries. This data is currently stored in a JSON file, which we will load to build our data structure.

References

1. University of Toronto Faculty of Arts & Science. “2025–2026 Academic Calendar.” *University of Toronto*, 2025, artsci.calendar.utoronto.ca/. Accessed 3 Mar. 2025.
2. Plotly Technologies Inc. “Plotly Python Open Source Graphing Library.” *Plotly*, 2025, plotly.com/python/. Accessed 3 Mar. 2025.
3. Plotly Technologies Inc. “Network Graphs in Python.” *Plotly*, 2025, plotly.com/python/network-graphs/. Accessed 3 Mar. 2025.
4. Richardson, Leonard. “Beautiful Soup Documentation.” *Crummy*, 2025, www.crummy.com/software/BeautifulSoup/bs4/doc/. Accessed 3 Mar. 2025.